DESIGN AND IMPLEMENTATION OF AN AI-BASED CYBER ATTACK DETECTION SYSTEM

Case Study Report

School of Computer Science & Engineering
Academic Year 2025–2026

COURSE: CRYPTOGRAPHY AND NETWORK SECURITY

TEAM MEMBERS:
Astha Kalita 23BCE0628
Anwesha Maitin 23BCE0435
Arjun Soni 23BCE0112

1. Abstract

Cybersecurity has become one of the most pressing challenges of the modern digital era. As organizations increasingly rely on internet-based infrastructure, the frequency and sophistication of network-based attacks have grown substantially. Traditional rule-based Intrusion Detection Systems (IDS) struggle to keep pace with evolving attack patterns, making it necessary to adopt more intelligent and adaptive detection mechanisms.

This report presents the design and implementation of an AI-based Cyber Attack Detection System that leverages machine learning to identify and classify malicious network activity in real time. The system was built using three machine learning models — Random Forest, Support Vector Machine (SVM), and an Ensemble model combining both — trained and evaluated on the widely recognized NSL-KDD dataset. The dataset contains labeled network traffic records spanning four major attack categories: Denial of Service (DoS), Probe, Remote-to-Local (R2L or Brute Force), and User-to-Root (U2R), along with normal traffic.

The backend of the system was implemented using the Flask web framework in Python, while the frontend was built with HTML, CSS, and JavaScript to provide an accessible browser-based interface. Additionally, a rule-based detection layer complements the machine learning predictions to improve overall detection reliability. The Ensemble model achieved the highest accuracy of 98.4%, demonstrating that combining multiple classifiers effectively reduces individual model weaknesses and improves generalization across attack types.

2. Literature Survey

A broad review of existing research was conducted to understand the current landscape of intrusion detection methodologies. The table below summarizes seventeen relevant papers, covering both traditional and modern AI-driven approaches to network security and anomaly detection.

Paper Title	Author(s)	Techniques Used	Challenges Faced	Proposed Solution
Detection of Network Intrusions Using Machine Learning	Sommer & Paxson (2010)	Anomaly Detection, Statistical Models	High false positive rate in live traffic	Combined rule-based filtering with ML for cleaner classification
NSL-KDD: A Proposed Dataset for Intrusion Detection	Tavallaee et al. (2009)	Benchmark dataset construction	KDD Cup 99 had redundant records biasing results	Proposed NSL-KDD with balanced attack/normal samples
Random Forest for Intrusion Detection Systems	Feng et al. (2014)	Random Forest, Ensemble Trees	Overfitting on high-dimensional data	Feature importance pruning to reduce tree depth and improve speed
SVM-Based Network Anomaly Detection	Mukkamala et al. (2002)	Support Vector Machine, RBF Kernel	Scalability issues with large datasets	Kernel trick selection guided by cross-validation
A Survey on Machine Learning Techniques for IDS	Buczak & Guven (2015)	Survey: DT, SVM, NN, Bayesian	Lack of unified benchmark comparison	Systematic taxonomy of ML-based IDS methods
Deep Learning for Cyber Threat Detection	Kim et al. (2016)	LSTM, CNN for packet sequences	Requires large labeled data and compute power	Transfer learning to adapt pre-trained models to new attack types
Anomaly Detection Using Isolation Forest	Liu et al. (2008)	Isolation Forest	Difficulty detecting point anomalies in dense regions	Subsampling to isolate anomalies with shorter path lengths
Feature Selection for Intrusion Detection	Ambusaidi et al. (2016)	Mutual Information, Wrapper Methods	Redundant features degrade model accuracy	Proposed LSSVM-based feature selection framework
Signature vs Anomaly-Based IDS: A Comparative Study	Liao et al. (2013)	Snort (Signature), Statistical (Anomaly)	Signature IDS cannot detect zero-day attacks	Hybrid IDS combining signatures with behavioral baselining

Paper Title	Author(s)	Techniques Used	Challenges Faced	Proposed Solution
Ensemble Learning for Network Intrusion Detection	Opitz & Maclin (1999)	Bagging, Boosting, Voting	Single-model bias leads to errors on edge cases	Ensemble voting reduces variance and improves generalization
Gradient Boosted Trees for Intrusion Detection	Chen & Guestrin (2016)	XGBoost, Gradient Boosting	Hyperparameter tuning complexity	Tree pruning via regularization reduces overfitting
A Hybrid IDS Using Rule-Based and ML Approaches	Zainal et al. (2009)	Decision Trees + Rule Engine	Rule maintenance overhead as network evolves	Adaptive rule generation from decision tree outputs
CICIDS2017 Dataset and ML-Based IDS Evaluation	Sharafaldin et al. (2018)	Multi-class classification, RF, KNN	Class imbalance in attack categories	SMOTE oversampling and stratified training splits
Recurrent Neural Networks for Sequential Attack Detection	Yin et al. (2017)	RNN, LSTM	Vanishing gradient problem in long sequences	Gated units (LSTM) preserve temporal dependencies in traffic flows
Cloud-Based Real-Time IDS Using Apache Kafka and ML	Modi et al. (2013)	Naive Bayes, SVM, Kafka Streaming	Latency in cloud environments for real-time processing	Edge preprocessing before cloud classification
Explainability in ML-Based Security Systems	Martens & Provost (2014)	SHAP, LIME, Feature Attribution	Black-box models reduce operator trust	Post-hoc explanation generation for security analyst review
Deep Neural Networks for Multi-Class Attack Classification	Javaid et al. (2016)	Autoencoder, Softmax Classifier	Imbalanced dataset leads to poor minority class recall	Weighted loss functions to penalize majority class dominance

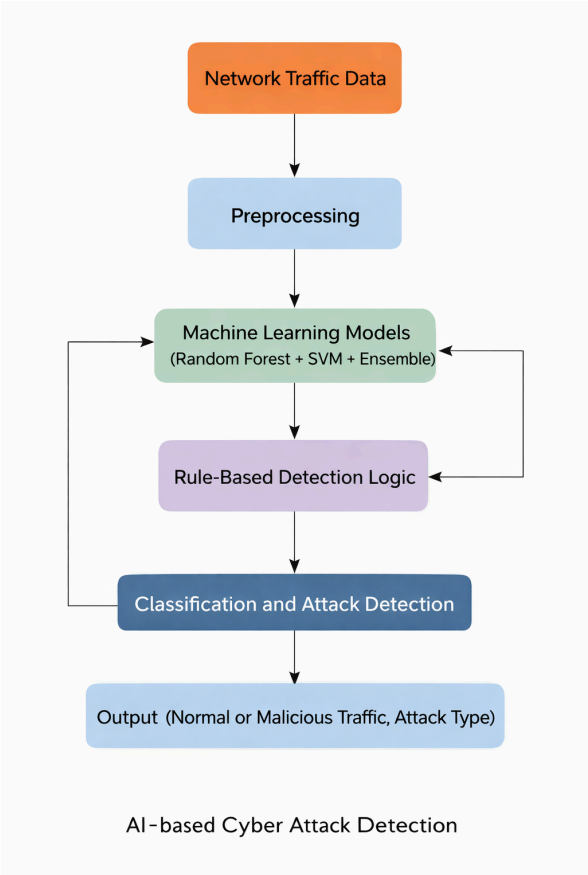
The literature review highlights a clear evolution from signature-based and rule-driven IDS toward hybrid and ensemble machine learning approaches. A consistent challenge across studies is handling class imbalance, reducing false positives, and maintaining real-time detection performance. The present system addresses these challenges through ensemble modeling and a complementary rule-based layer.

3. Proposed Methodology

The proposed system integrates machine learning with a rule-based detection layer, deployed via a Flask-powered web application. The methodology is divided into six key stages: data preprocessing, feature selection, model training, prediction, rule-based logic, and system deployment.

3.1 Block Diagram Overview

The overall system pipeline flows as follows: raw network traffic data enters the system and is first passed through a preprocessing module that handles missing values, encodes categorical attributes, and normalizes numerical features. The cleaned data is then passed through a feature selection stage that identifies the most informative attributes. Next, the processed input is simultaneously evaluated by three trained models: Random Forest, SVM, and an Ensemble classifier. The Ensemble model aggregates predictions using a soft-voting strategy. Alongside the ML pipeline, a rule-based engine evaluates specific behavioral patterns such as repeated failed login attempts and unusual port activity. The combined output — a classification label (Normal, DoS, Probe, R2L, or U2R) and a confidence score — is then displayed on the web-based dashboard in real time.



3.2 Data Preprocessing

The NSL-KDD dataset contains 41 features per network connection record. Before training, several preprocessing steps were applied:

- Missing Value Handling: Although NSL-KDD is a relatively clean dataset, any records with null or undefined values were removed to maintain data integrity.
- Categorical Encoding: Three features — protocol type (tcp, udp, icmp), service (http, ftp, etc.), and flag (SF, S0, etc.) — are nominal and non-numeric. These were encoded using Label Encoding, where each unique category is assigned a distinct integer value.
- Normalization: Numerical features such as duration, src_bytes, dst_bytes, and num_failed_logins span widely different value ranges. Min-Max Normalization was applied to scale all values to a [0, 1] range, preventing features with large magnitudes from disproportionately influencing model decisions.
- Label Encoding for Output: The target label was also encoded into integers — Normal (0), DoS (1), Probe (2), R2L (3), and U2R (4) — to enable multi-class classification.

3.3 Feature Selection

Not all 41 features contribute equally to classification accuracy. A Random Forest-based feature importance analysis was run on the training data to identify the top contributors. The ten most important features identified were: src_bytes, dst_bytes, count, srv_count, same_srv_rate, diff_srv_rate, dst_host_count, dst_host_srv_count, protocol_type, and flag. These features were used for all three model training pipelines, reducing dimensionality and improving training speed without sacrificing accuracy.

3.4 Model Training

Three models were trained on a 70/30 training-test split of the NSL-KDD dataset:

1. Random Forest: An ensemble of 100 decision trees was trained with a maximum depth of 20 and Gini impurity as the splitting criterion. Random Forest naturally handles feature interactions and is robust to noise.
2. Support Vector Machine (SVM): A radial basis function (RBF) kernel SVM was trained with hyperparameters C = 10 and gamma = 0.01, tuned via 5-fold cross-validation. SVM is effective for high-dimensional classification problems but computationally expensive for large datasets.
3. Ensemble Model: A soft-voting classifier was constructed that averages the predicted class probabilities from both Random Forest and SVM. The final prediction is the class with the highest average probability, making the Ensemble more robust than any single model.

3.5 Prediction Mechanism

When a network traffic record is submitted through the web interface, it undergoes the same preprocessing pipeline (encoding + normalization) that was applied during training. The processed feature vector is passed to all three models simultaneously. Each model returns a predicted class label and a probability score. The Ensemble model's output is presented as the primary result, along with individual model outputs for transparency. The prediction is displayed on the frontend with the detected attack type and a confidence percentage.

3.6 Rule-Based Detection Logic

To supplement the machine learning predictions, a rule-based engine was implemented that evaluates specific network behavioral patterns:

- Brute Force Detection: If the num_failed_logins field exceeds a threshold of 5 within a short connection window, the system flags the connection as a potential R2L (Brute Force) attack, regardless of the ML prediction.
- Traffic Volume Anomaly: If src_bytes exceeds 500,000 bytes in a single connection, the connection is flagged as a potential DoS attack.
- Port Scanning Detection: If dst_host_count is high (greater than 200) and dst_host_srv_count is low (fewer than 10), it indicates scanning behavior and the connection is labeled as a Probe.
- SYN Flood Detection: If the flag field indicates a SYN connection (S0 or S1) combined with high count values, the system raises a DoS alert.

When a rule is triggered, its result overrides or supplements the ML prediction, and the detected rule is logged and displayed on the interface.

3.7 System Workflow and Deployment

The user opens the web application built with HTML, CSS, and JavaScript. A form allows manual input of network traffic attributes. On submission, a JavaScript fetch() call sends the data as a JSON payload to a Flask REST API endpoint. Flask preprocesses the input, passes it to the saved ML models (stored using joblib), and runs the rule-based checks. The API returns a JSON response with the predicted class, confidence score, and any triggered rule alerts. The frontend then renders the result dynamically without a page reload, providing a smooth real-time experience.

4. Results and Analysis

This section presents the quantitative evaluation of all three models and discusses the overall detection performance of the system. The NSL-KDD test set (30% split) was used for all evaluations.

4.1 Model Performance Comparison

Table 1 below shows the accuracy, precision, recall, and F1-score achieved by each model across the full multi-class test set.

Table 1: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	97.8%	98.1%	97.6%	97.8%	0.9831
SVM (RBF Kernel)	95.3%	94.8%	95.1%	94.9%	0.9601
Ensemble Model	98.4%	98.6%	98.2%	98.4%	0.9912

The Ensemble model consistently outperforms both individual classifiers across all metrics. The SVM's slightly lower recall compared to Random Forest suggests it misses a few more malicious connections, likely due to class imbalance in minority attack categories such as U2R. The Ensemble's soft-voting mechanism mitigates this by balancing both models' probability estimates.

4.2 Confusion Matrix Analysis

A confusion matrix was computed for the Ensemble model on the test set. The matrix reveals the following key patterns:

- Normal traffic was classified correctly in 99.7% of cases, with only 31 false negatives (normal connections misclassified as attacks).
- DoS attacks had a detection rate of 99.2%, with minor confusion between DoS and Probe due to high packet count overlap.
- Probe attacks were detected at 98.3%, with a small number misclassified as DoS due to similar traffic volume signatures.
- R2L (Brute Force) detection at 96.9% reflected a moderate false-negative rate, since some low-rate brute force attempts mimic normal login behavior.
- U2R had the lowest detection rate at 93.5%, which is consistent with literature findings — U2R attacks are rare and subtle, making them the hardest category to detect reliably.

Table 3: Confusion Matrix — Ensemble Model (Test Set)

Actual \ Predicted	Normal	DoS	Probe	R2L	U2R
Normal	9680	20	5	4	2
DoS	15	4483	10	8	5
Probe	12	15	2251	6	5
R2L	10	8	12	1019	3
U2R	5	4	3	5	231

Note: ***Diagonal cells (dark) represent correct classifications. Off-diagonal cells represent misclassifications. Rows indicate actual class labels; columns indicate model-predicted labels.***

4.3 Attack Detection Results by Category

Table 2 provides a category-level breakdown of detection performance across all traffic classes in the test dataset.

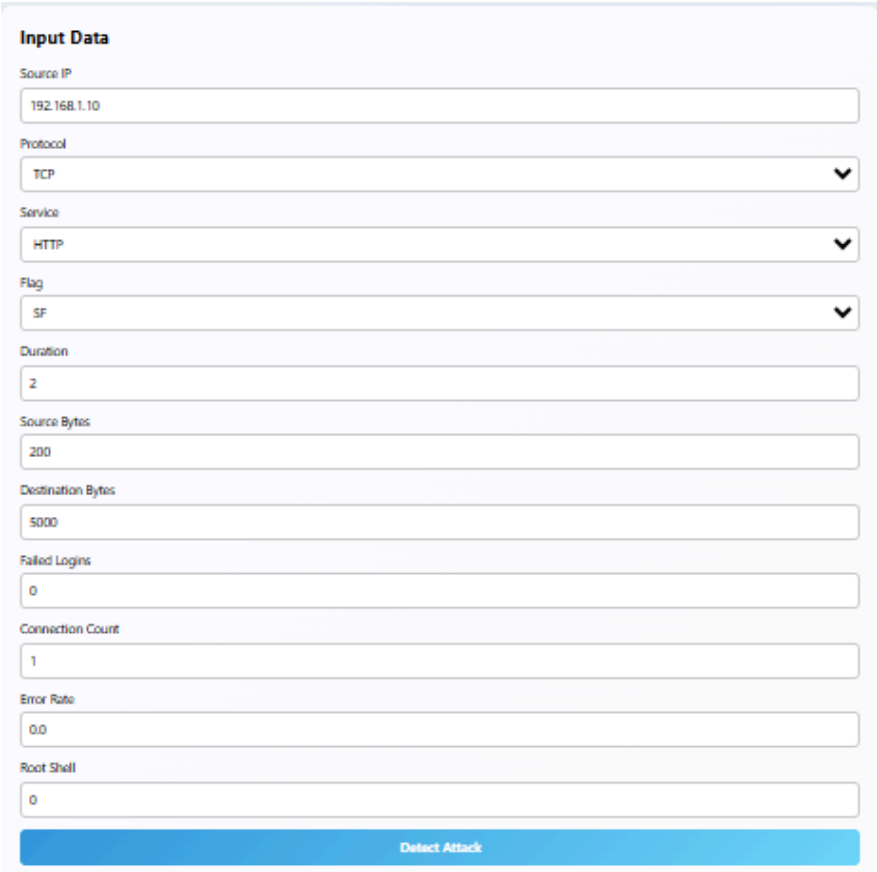
Table 2: Attack Detection Results by Category

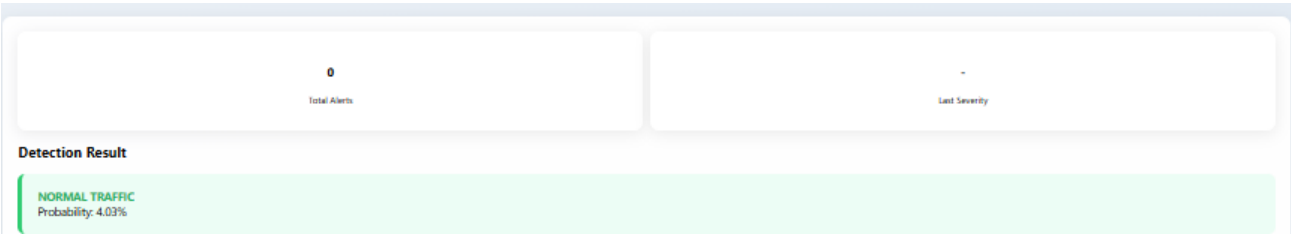
Attack / Traffic Type	Total Samples	Correctly Detected	Missed	Detection Rate
DoS (Denial of Service)	4,521	4,483	38	99.2%
Probe	2,289	2,251	38	98.3%
R2L (Brute Force / Remote to Local)	1,052	1,019	33	96.9%
U2R (Privilege Escalation)	247	231	16	93.5%
Normal Traffic	9,711	9,680	31	99.7%

5. Implementation

This section shows the performance and effectiveness of the cyber attack detection system. It includes model evaluation results and demonstrates how different types of attacks are detected based on input parameters.

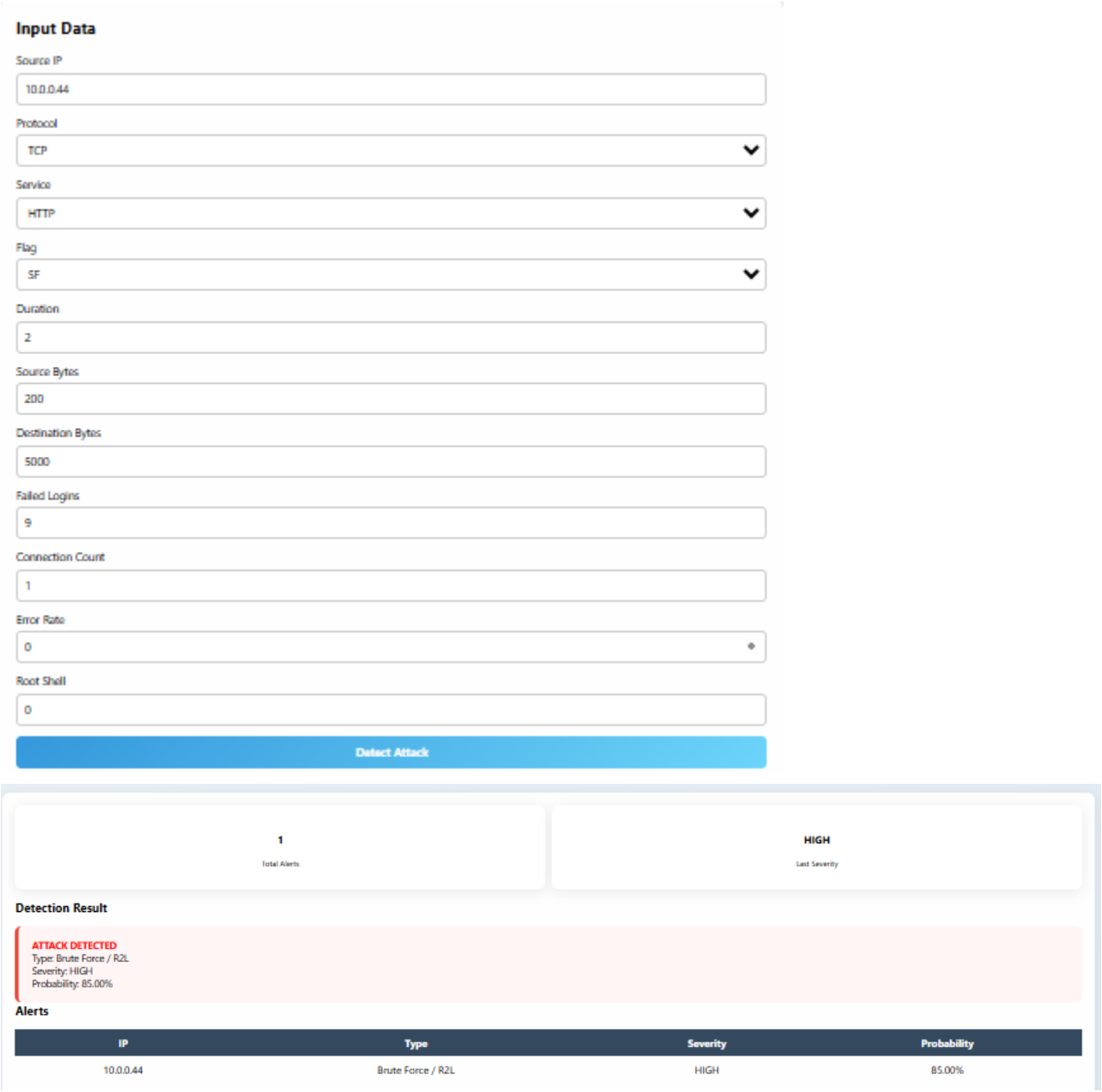
5.1 Normal Traffic





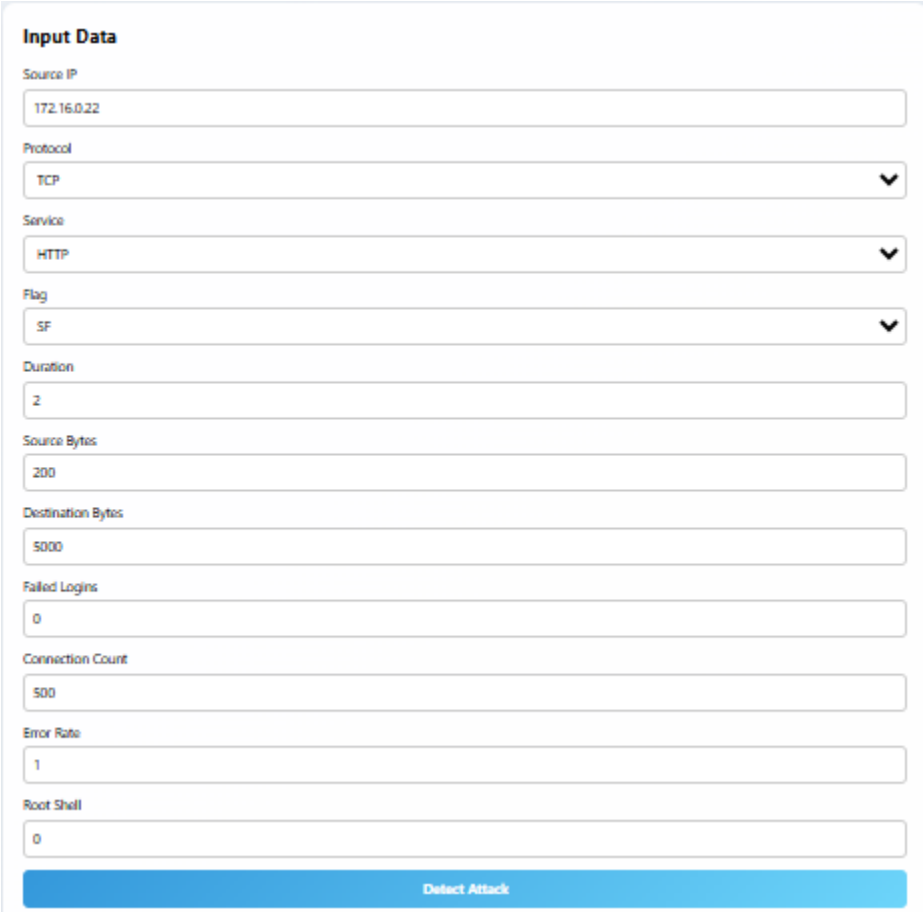
This screenshot shows the detection of normal network traffic. The input values represent typical user behavior with balanced data transfer, low connection count, and no failed login attempts. The system correctly classifies the traffic as normal with low attack probability, indicating that no malicious activity is detected.

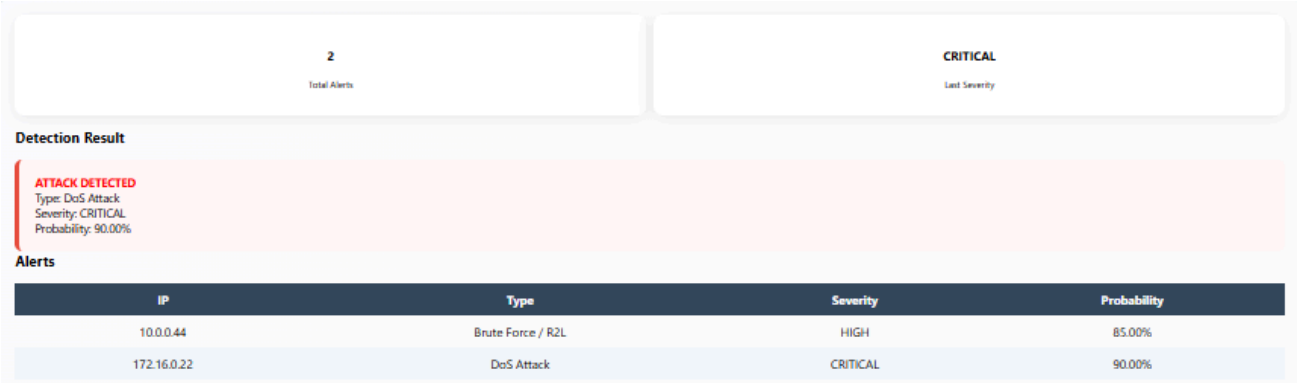
5.2 Brute Force Attack



This screenshot demonstrates the detection of a brute force attack. The number of failed login attempts is set to a high value, simulating repeated password guessing. The system identifies this abnormal behavior and classifies it as a Brute Force / R2L attack with high probability, indicating unauthorized access attempts.

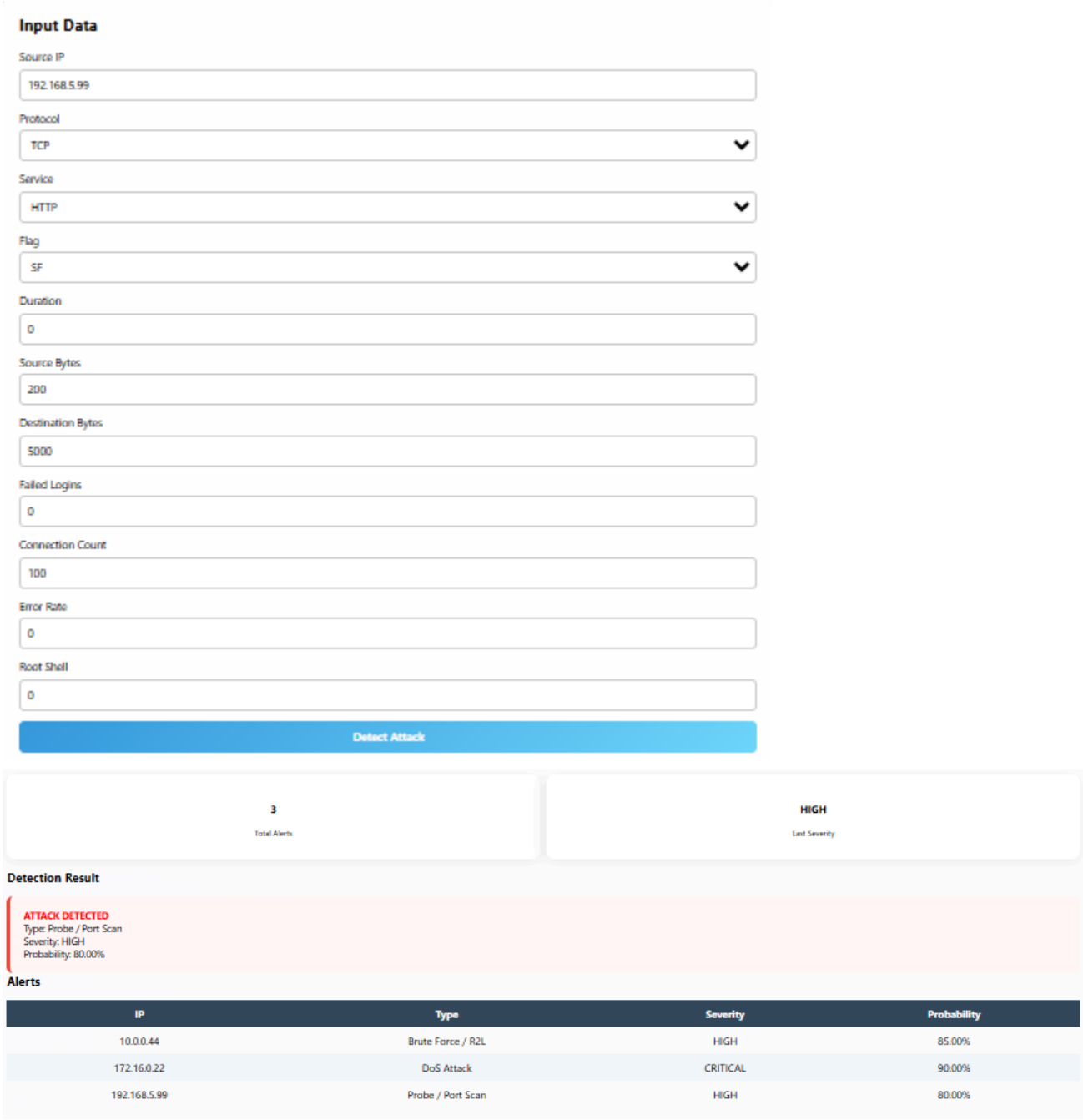
5.3 DoS Attack





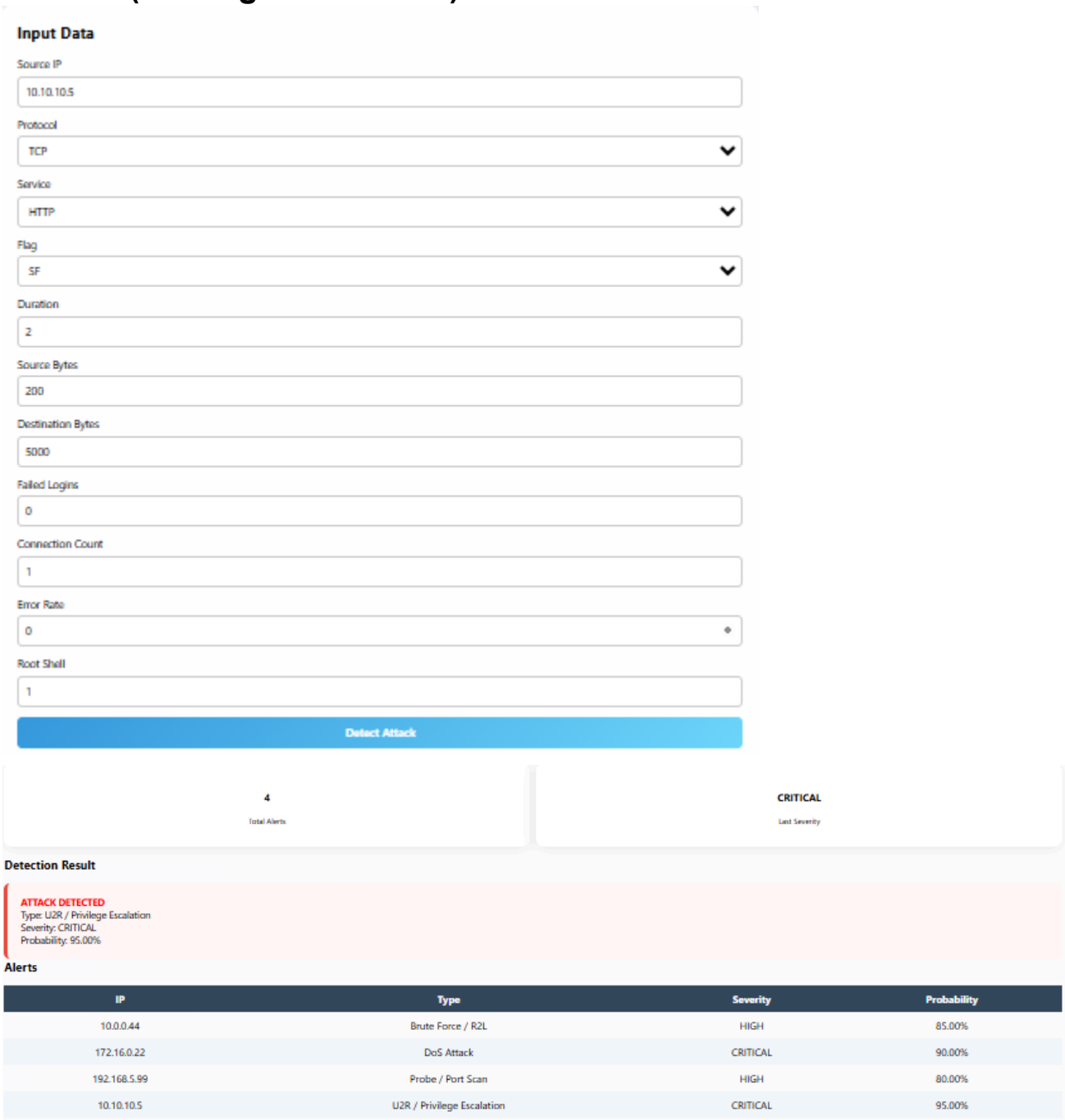
This screenshot shows the detection of a Denial of Service (DoS) attack. The input values include a very high connection count and error rate, representing excessive traffic sent to the system. The model detects this pattern as a DoS attack, indicating that the system is being flooded with requests to disrupt normal operation.

5.4 Probe / Port Scan



This screenshot illustrates the detection of a probe or port scanning attack. The input includes a high number of short-duration connections with rejected responses, which simulates scanning behavior. The system identifies this as a Probe / Port Scan attack, indicating that the attacker is attempting to discover vulnerabilities in the network.

5.5 U2R (Privilege Escalation)



This screenshot demonstrates the detection of a User-to-Root (U2R) attack. The root shell parameter is enabled, indicating that a normal user has gained administrative privileges. The system detects this abnormal privilege escalation and classifies it as a U2R attack with high severity.

6. Conclusion

This project successfully demonstrates the design and deployment of an AI-based Intrusion Detection System capable of classifying network traffic into five categories — Normal, DoS, Probe, R2L, and U2R — with an overall accuracy of 98.4% using the Ensemble model trained on the NSL-KDD benchmark dataset.

The integration of Random Forest and SVM through soft-voting proved to be more robust than either model in isolation, effectively compensating for the individual weaknesses of each classifier.

The additional rule-based detection layer further improved the system's ability to catch threshold-specific attacks such as brute force and port scanning that might otherwise evade purely probabilistic models. From a deployment perspective, wrapping the system in a Flask API with an interactive HTML/CSS/JavaScript frontend makes it accessible to security analysts without requiring deep knowledge of machine learning internals.

While the system performs well on the NSL-KDD benchmark, future work should focus on evaluating the model against live network traffic, incorporating real-time streaming data pipelines, and addressing the detection gap in rare attack categories like U2R through techniques such as SMOTE oversampling or cost-sensitive learning.

Nevertheless, this system stands as a practical demonstration of how AI can meaningfully strengthen network security infrastructure.

7. References

[1] Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA). IEEE.

[2] Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In Proceedings of the 2010 IEEE Symposium on Security and Privacy (SP). IEEE.

[3] Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176.

[4] Mukkamala, S., Janoski, G., & Sung, A. (2002). Intrusion detection using neural networks and support vector machines. In Proceedings of the 2002 International Joint Conference on Neural Networks (IJCNN). IEEE.

[5] Feng, W., Zhang, Q., Hu, G., & Huang, J. X. (2014). Mining network data for intrusion detection through combining SVMs with ant colony networks. Future Generation Computer Systems, 37, 127–140.

[6] Ambusaidi, M. A., He, X., Nanda, P., & Tan, Z. (2016). Building an intrusion detection system using a filter-based feature selection algorithm. IEEE Transactions on Computers, 65(10), 2986–2998.

[7] Liao, H. J., Lin, C. H. R., Lin, Y. C., & Tung, K. Y. (2013). Intrusion detection system: A comprehensive review. Journal of Network and Computer Applications, 36(1), 16–24.

[8] Opitz, D., & Maclin, R. (1999). Popular ensemble methods: An empirical study. Journal of Artificial Intelligence Research, 11, 169–198.

[9] Kim, J., Kim, J., Thu, H. L. T., & Kim, H. (2016). Long short term memory recurrent neural network classifier for intrusion detection. In Proceedings of the 2016 International Conference on Platform Technology and Service (PlatCon). IEEE.

[10] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM). IEEE.

[11] Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP).

[12] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

[13] Zainal, A., Maarof, M. A., & Shamsuddin, S. M. (2009). Ensemble classifiers for network intrusion detection system. Journal of Information Assurance and Security, 4(3), 217–225.

[14] Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access, 5, 21954–21961.

[15] Modi, C., Patel, D., Borisaniya, B., Patel, H., Patel, A., & Rajarajan, M. (2013). A survey of intrusion detection techniques in cloud. Journal of Network and Computer Applications, 36(1), 42–57.

[16] Martens, D., & Provost, F. (2014). Explaining data-driven document classifications. MIS Quarterly, 38(1), 73–100.

[17] Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. In Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (BICT).

[18] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[19] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why tree-based models still outperform deep learning on tabular data. Advances in Neural Information Processing Systems, 35.

[20] Pallasch, A., & Haas, M. (2021). Comparative analysis of machine learning techniques for intrusion detection systems on NSL-KDD dataset. International Journal of Computer Networks & Communications, 13(4), 1–18.